

STUDENT EXPENSE TRACKER

A Comprehensive Python & SQLite Project Report

****Project Type:** Python & SQLite Console Application**

****Programming Language:** Python 3.x**

****Database Engine:** SQLite3**

****User Interface:** Command-Line Interface (CLI)**

****Primary Purpose:** Personal Expenditure Monitoring, Budget Governance, and Financial Analytics for Students**

Student Information

*** **Name:** Prisha Singh**

*** **Registration number:** 26BCE11343**

*** **Course / Class:** B.Tech CSE Core**

*** **Date of Submission:** 29-09-2026**

1. Abstract

The ****Student Expense Tracker**** is a lightweight, console-based Python application designed to help students systematically record, manage, and analyze their daily expenditures. Built with a focus on ease of use and structural modularity, the application utilizes an embedded SQLite database for secure and persistent data storage. Through an intuitive, me

nu-driven interface, users can perform comprehensive CRUD (Create, Read, Update, Delete) operations on their expense records.

Beyond simple logging, the application incorporates a predictive budget-monitoring subsystem that calculates aggregated expenditures against user-defined monthly allowances. To encourage proactive financial discipline, the system triggers automated alerts when spending approaches critical thresholds (e.g., reaching 80% or exceeding the total budget). To ensure long-term maintainability, the project's source code is organized into distinct functional modules, decoupling data access layers from business logic and user interface components.

2. Project Objectives

The overarching goal of this project is to build a practical tool that fosters financial literacy and responsibility among students. The specific engineering and functional objectives include:

- * **Persistent Data Architecture:** Design a robust local storage mechanism using SQLite to ensure data continuity across application sessions.

- * **Comprehensive Record Lifecycle Management:** Establish reliable workflows for capturing, viewing, updating, and

removing individual expense entries.

Automated Financial Calculation: Programmatic calculation of aggregated real-time expenditures from the database layer.

Categorized Spending Intelligence: Group expenses dynamically into distinct domains—such as *Food*, *Transport*, *Education*, *Entertainment*, *Shopping*, and *Other*—to highlight spending habits.

Proactive Budgetary Governance: Allow users to set hard spending caps and receive situational warnings when historical expenses hit or breach 80% and 100% thresholds.

Modular Software Engineering: Avoid monolithic script architecture by separating the application into specialized modules focused on database drivers, business logic, analytics, and presentation.

3. Functional Features

The application exposes its capabilities through a clean, number-driven console menu. The table below outlines the core functional options available to the user:

Option	Feature	Technical Description
:---	:---	:---
1	**Add Expense**	Captures input parameters (Amount, Category, Description) and automatically timestamps th

e transaction using system time. |

| ****2**** | ****View Expenses**** | Fetches and renders all saved transactions in a clean tabular layout, showing IDs, amounts, categories, notes, and dates. |

| ****3**** | ****Total Spent**** | Aggregates all recorded spending to provide an immediate summary of global expenditures. |

| ****4**** | ****Category Analysis**** | Performs a SQL grouping operation to summarize and display cumulative spending isolated by specific categories. |

| ****5**** | ****Delete Expense**** | Permanently drops an expense record from the persistent database using its unique Primary Key (ID). |

| ****6**** | ****Edit Expense**** | Updates any field of an existing record (Amount, Category, Description, Date) based on its unique transaction ID. |

| ****7**** | ****Set Budget**** | Overwrites or initializes the active user's monthly spending limit after running comprehensive numerical validations. |

| ****8**** | ****Check Budget**** | Provides an analytical breakdown of total budget, spending to date, remaining cash, and active warning flags. |

| ****9**** | ****Exit**** | Gracefully closes database connections, disposes of active cursors, and terminates the application loop. |

4. Technologies and Tools Used

The development of this system relies exclusively on modern, standard-compliant technologies that ensure high performance in standard desktop environments:

- * **Python:** Serves as the core programming language, handling execution flow, console input/output parsing, and mathematical evaluations.

- * **SQLite:** Provides an embedded, zero-configuration relational database engine that stores application data directly into a single system file.

- * **sqlite3 (Python Standard Library):** Acts as the database driver, establishing connections, executing parameterized SQL statements, and retrieving result sets.

- * **datetime (Python Standard Library):** Manages temporal data tracking, ensuring every log entry captures highly accurate date metadata.

- * **Visual Studio Code (VS Code):** Used as the Integrated Development Environment (IDE) for coding, debugging, version management, and local terminal execution.

5. System Design and File Structure

To adhere to the principle of Separation of Concerns (SoC), the application's source code is distributed across several distinct Python scripts. This modular approach ensures that m

odifications to the database engine or analytics formulas do not compromise user interface routines.

`***`main.py`**`

***Responsibility:** The entry point of the software. It handles the continuous user interface loop, displays menu layouts, captures user input, and routes commands to appropriate modules.

`***`database.py`**`

***Responsibility:** Manages structural persistence. It establishes connections to the database file and ensures the relational schema exists upon initial startup.

`***`expense_manager.py`**`

***Responsibility:** Contains operational logic for managing individual records, including creating, querying, modifying, and deleting expense records.

`***`budget_manager.py`**`

***Responsibility:** Governs budgeting operations, storing active limits, evaluating margins, and raising threshold warnings.

`***`analytics.py`**`

***Responsibility:** Handles statistical aggregation, processing underlying data sets to compile category summaries.

`***`expenses.db`**`

***Responsibility:** The persistent binary file created by SQLite containing the structured data tables.

6. Database Schema Design

The application keeps a streamlined schema design, using a singular structured table named `expenses`. This setup minimizes query latency while matching all requirements.

Table: `expenses`

Column Name	Data Type	Key Type	Constraint / Description
---	---	---	---
id	INTEGER	PRIMARY KEY	Auto-incrementing unique identifier assigned to every single transaction.
amount	REAL	None	Floating-point value representing the financial cost of the item/service.
category	TEXT	None	Categorical label matching one of the predefined application domains.
description	TEXT	None	Contextual notes or explanations written by the user regarding the expense.
date	TEXT	None	ISO 8601 formatted text representation (`YYYY-MM-DD`) tracking when the log occurred.

Schema Initialization Statement:

```
```sql
```

```
CREATE TABLE IF NOT EXISTS expenses (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 amount REAL NOT NULL,
```

```

category TEXT NOT NULL,
description TEXT,
date TEXT NOT NULL
);

```

```

```

## ## 7. Core Program Logic and Workflows

### ### 7.1 Record Injection (Adding an Expense)

When a user logs a transaction, the system reads raw data from the console, maps the chosen category index to a human-readable string, and references the `datetime` module for the current date stamp. To protect the database from SQL injection vulnerabilities, the application strictly uses parameterized queries.

```

```python
# Conceptual Database Insertion Workflow
cursor.execute("""
    INSERT INTO expenses (amount, category, description, date)
    VALUES (?, ?, ?, ?)
""", (amount, category, description, date))

```

7.2 Financial Aggregation (Calculating Total Expenditure)

Rather than pulling datasets into volatile memory and calculating totals inside Python loops, the application shifts computation directly to the database engine via optimized aggregate functions. This approach ensures high data integrity and performance.

```
```sql
SELECT SUM(amount) FROM expenses;
```
```

7.3 Budgetary Risk Evaluation

The budget evaluation workflow compares real-time aggregate costs against user-defined boundaries. The application dynamically determines the remaining balance and flags potential issues using the following logic:

$$\text{Remaining Funds} = \text{Monthly Budget} - \text{Total Spent}$$

Condition A (Warning State): If $\text{Total Spent} \geq (0.80 \times \text{Budget})$ and $\text{Total Spent} < \text{Budget}$, a cautionary alert warns the user that 80% or more of their allowance has been consumed.

Condition B (Over-Budget State): If $\text{Total Spent} \geq \text{Budget}$, an urgent over-budget alert warns the user that their budget ceiling has been breached.

8. Robust Input Validation and Error Handling

To guarantee runtime resilience, the system implements defensive programming strategies that intercept invalid runtime inputs before they can trigger unhandled exceptions or corrupt data stores:

Numerical Parsing Defenses: All currency entries for expenses and budget allocations are filtered through `try/except ValueError` blocks, filtering out alpha-numeric typos, blank inputs, and negative coefficients.

Categorical Bounds Verification: Menu selections and expenditure categories restrict choices to precise integer arrays (e.g., `1` through `6`). Entries outside these boundaries are caught and rejected.

Null Reference Interception: When the `expenses` database table is entirely empty, the standard SQL `SUM()` operation evaluates to `NULL`. The system catches this occurrence and safely coerces it to `0` or `0.00` to prevent math errors.

State Check Assertions: The system checks if a budget limit exists before performing comparison math, prompting users to establish a budget limit first if none is found.

Identity Auditing: Prior to attempting modifications or removals on an existing record, the system verifies the user-supplied transaction ID exists within the database table, reporting clear missing errors instead of crashing.

9. Testing Matrix and Quality Assurance

System validation was conducted through structured manual test suites targeting every operational menu choice. The results prove the stability of the console implementation:

| Test Reference | Feature Under Test | Input Vector / Actions | Expected Behavior & Verified Outcome | Status |
|----------------|------------------------|--|--|------------|
| :--- | :--- | :--- | :--- | :--- |
| **TC-01** | Transaction Generation | Valid entry parameters provided | Record injected successfully; confirmation prompt shown. | **Passed** |
| **TC-02** | Tabular Data Query | Choose option to view logs | All saved records rendered perfectly in structural rows. | **Passed** |
| **TC-03** | Summation Engine | Trigger global expenditure calculation | Sum accurately computed by SQLite engine and output to terminal. | **Passed** |
| **TC-04** | Statistical Breakdown | Execute category analytics | Costs accurately partitioned and summarized by labels. | **Passed** |
| **TC-05** | Record Dropping | Provide an active record ID | Selected entry removed from database; display tables updated. | **Passed** |
| **TC-06** | Record Amendment | Target an active record ID with new data | Changes directly applied to targeted database row columns. | **Passed** |

| **TC-07** | Budget Entry | Input positive floating-point value | Allocation accepted and written to system memory. | **Passed** |

| **TC-08** | Budget Audit (Alerts) | Query budget states at 85 % expenditure | Displays full accounting alongside warning flags. | **Passed** |

| **TC-09** | Empty-State Resilience | Check budget status prior to initializing allocation | System detects lack of configuration; prompts configuration. | **Passed** |

10. Engineering Challenges and Optimization Solutions

| Encountered Challenge | Technical Root Cause | Adopted Architectural Solution |

| :--- | :--- | :--- |

| **Data Inconsistency Across Files** | Separating the code into files made it hard to keep tracking totals accurate across different code modules. | Refactored total generation into an isolated database call `get\_total\_spent()`, ensuring budget managers always run queries on live tables. |

| **Uninitialized App States** | Evaluating empty or unconfigured variables threw errors when testing budget positions. | Introduced logical safety guards that verify the state of variables before checking spending comparisons. |

| **Monolithic Code Growth** | Early iterations grouped all code paths together, making updates difficult and confusing.

| Applied single-responsibility design patterns to divide code into specialized modules (`database`, `expense\_manager`, `budget\_manager`, `analytics`). |

| **Terminal Crash Risk** | Users typing non-numeric characters into numeric entry forms triggered unhandled program failures. | Wrapped all terminal inputs in comprehensive `try/except` constructs combined with conditional numerical range validation checks. |

11. Project Limitations and Future Roadmap

While the current working iteration perfectly achieves its baseline requirements, it operates within clear functional boundaries. The roadmap below charts future planned updates:

Graphical User Interface (GUI): Transition away from terminal interfaces towards visual layouts built with libraries like `Tkinter` or `PyQt`.

Permanent Configurations: Shift budget configuration targets into a separate configuration table inside `expenses.db` so configurations persist across system reboots.

Advanced Query Filters: Introduce customized searching, enabling users to isolate expenditures across custom date frames or categories.

Visual Analytical Charts: Integrate plotting frameworks such as `matplotlib` to render beautiful, colorful spending pie charts and trend charts.

* **Data Portability Pipelines:** Write data export engines capable of compiling local records into universally compatible format types like `.csv` or `.xlsx`.

* **Multi-User Security Profiles:** Integrate encrypted local databases alongside user credential authorization panels to support multiple user profiles safely.

12. Conclusion

The development of the **Student Expense Tracker** highlights the practical application of fundamental computer science concepts and software engineering patterns. By combining structured programming practices, input protection, and a modular architecture with an embedded relational database engine, the project provides a responsive and reliable financial companion tool for students.

The successful implementation of modular decoupling serves as an excellent foundation for future feature expansions, ensuring the system remains extensible. Testing confirms that the system handles edge cases gracefully, maintaining excellent runtime safety across all operational scenarios.

13. References and Resources Used

* **Python Documentation:** Standard library reference guides covering `sqlite3` and `datetime` frameworks.

* **SQLite Engineering Manual:** Relational database query construction syntax guidelines.

* **Visual Studio Code Environment:** Extensively used for writing, linting, executing, and testing codebase builds.